

Practice Sheet 2 : Lambda Expressions, Method References and Enumeration

Exercise 1: Implement different approaches to compute the square of an integer using Java's predefined functional interfaces (see the last page).

For each interface :

1. Declare a lambda expression that returns the square of an input.
2. Apply the lambda to test values and display the results.
3. Wrap all implementations inside a single main method.

Exercise 2: Given the following MyClass java class with StReverse static method that takes a string as an input parameter and returns its reverse :

```
1 class MyClass{
2     public static String StReverse(String str)
3     { String result = "";
4       for(int i = str.length()-1; i>=0 ; i--)
5           result + = str.charAt(i);
6       return result;
7     }
8 }
```

Listing 1 – class MyClass

In the main program

1. Test the method using a custom functional interface, a lambda expression, and tree types of method references.
2. Test the method using a predefined functional interface (see the last page).

Exercise 3:

- a) Define a custom functional interface called MathOperation with a method that takes two int parameters and returns an int.
- b) Declare an enum Operation where each constant represents an arithmetic operation (+, -, * and /).
- c) Associate each enum constant with its corresponding implementation.
- d) Define a method operate(Operation op, int a, int b) that applies the given operation to the two integers and returns the result.
- e) In the main program, lets the user to input an arithmetic operation,two integers and perform the selected operation and display the result.

Exercise 4: Given the following Person java class :

```
1  class Person {
2      private String name;
3      private int yearBirth;
4
5      public Person(String name, int yearBirth){
6          this.name = name;
7          this.yearBirth = yearBirth;
8      }
9
10     public String getName(){
11         return name;
12     }
13
14     public int getYearBirth(){
15         return yearBirth;
16     }
17
18     public String toString(){
19         return "Person[Name: " + name + ", yearBirth: " + yearBirth + "]"
20         ;
21     }
22 }
```

Listing 2 – class Person

Use **predefined functionnal interfaces** and **Methods reference** to implement the following statement :

- a) Method to check whether a person's name starts with an uppercase letter.
- b) Method that converts a person's name to uppercase.
- c) Method that return the person's age.

In the Main class, instantiate objects of the Person class (these objects are used to test the methods implemented above), then compare a set of persons by their name, and then by their age.

Note : its hadly recommended to implement tree sorting approaches : one with a Static inner class ,lambda exepression and another using method references .

Summary of Predefined Functional Interfaces in Java

Interface	Method	Description
<code>Predicate<T></code>	<code>boolean test(T t)</code>	Evaluates a condition on an input and returns true or false. Used for filtering.
<code>Consumer<T></code>	<code>void accept(T t)</code>	Performs an operation on an input without returning a result.
<code>Supplier<T></code>	<code>T get()</code>	Supplies a value without taking any input.
<code>Function<T, R></code>	<code>R apply(T t)</code>	Takes one argument and returns a result.
<code>BiFunction<T, U, R></code>	<code>R apply(T t, U u)</code>	Takes two arguments and returns a result.
<code>UnaryOperator<T></code>	<code>T apply(T t)</code>	A specialization of Function where input and output types are the same.
<code>BinaryOperator<T></code>	<code>T apply(T t1, T t2)</code>	A specialization of BiFunction for the same input and output types.
<code>BiPredicate<T, U></code>	<code>boolean test(T t, U u)</code>	Takes two inputs and returns a boolean. Often used for comparisons.
<code>BiConsumer<T, U></code>	<code>void accept(T t, U u)</code>	Takes two inputs and performs an operation without returning a result.
<code>IntConsumer</code>	<code>void accept(int value)</code>	Consumer for primitive <code>int</code> .
<code>IntSupplier</code>	<code>int getAsInt()</code>	Supplies a primitive <code>int</code> .
<code>IntPredicate</code>	<code>boolean test(int value)</code>	Tests a condition on a primitive <code>int</code> .
<code>IntUnaryOperator</code>	<code>int applyAsInt(int operand)</code>	Unary operator for <code>int</code> .
<code>IntBinaryOperator</code>	<code>int applyAsInt(int left, int right)</code>	Binary operator for two <code>ints</code> .
<code>ToIntFunction<T></code>	<code>int applyAsInt(T value)</code>	Converts an object of type <code>T</code> to a primitive <code>int</code> .
<code>ToIntBiFunction<T, U></code>	<code>int applyAsInt(T t, U u)</code>	Takes two arguments and returns an <code>int</code> .

For more information, refer to the official Java documentation : [Java 8 Functional Interfaces Documentation](#).